



FULL STACK WEB APPLICATION

- Video Testimonial Tool

Full Stack Web Development

Overview

The Video Testimonial Application is a web-based tool that allows users to record video testimonials directly from their browsers and upload them to Google Drive.

Technologies Used

Frontend:

HTML: Provides the structure of the web page.

index.html

CSS: Enhances the visual presentation with styles.

styles.css

JavaScript: Manages client-side functionality, including media capture and interaction with the backend.

JavaScript (app.js)

Media Capture: Uses the MediaDevices API to access the user's camera and microphone.

Recording Control: Manages recording start/stop and captures video chunks.

Upload: Sends the recorded video to the backend server for uploading to Google Drive.

Backend:

Node.js: Server-side runtime environment.

Express.js: Web framework for building the server and API endpoints.

Multer: Middleware for handling file uploads.

Multer is a middleware for handling multipart/form-data, which is primarily used for uploading files in Node.js applications.

It makes it easy to handle file uploads by parsing the incoming form data and attaching the files to the req.file or req.files object, depending on whether a single file or multiple files are being uploaded.

Google APIs: Client library for interacting with Google Drive API.

CORS: Middleware for enabling Cross-Origin Resource Sharing.

CORS is used to enable the frontend application (running in the browser) to communicate with the backend server running on a different origin

Server Setup (server.js)

#1

Install Packages

npm init -y

npm install express multer

npm install googleapis google-auth-library

#2

Google Cloud Project and Drive API Setup

Create or Select a Google Cloud Project

Visit

<https://console.cloud.google.com/>

Create a new project or select an existing one

Enable the Google Drive API

Navigate to the API Library: From the dashboard of your selected project, look for the "APIs & Services" dashboard, then click "Library".

Search for the Google Drive API: In the search bar, type "Google Drive API" and select it from the results

Enable the API: Click the "Enable" button to activate the Google Drive API for your project.

After enabling the API, you'll be prompted to create credentials to use the API. Click "Create Credentials" or go to the "Credentials" tab from the "APIs & Services" menu.

Click "Create credentials" and select "Service account" from the dropdown. Service accounts are used for server-to-server interactions without user intervention.

Provide a name for the service account.

You can optionally provide a description.

Click "Create".

Assign the service account a role. For uploading files to Drive, roles like "Editor" or specific Drive roles ("Drive File Stream" or "Drive API") can be used. Be cautious with permissions; grant the minimum necessary for your application to function.

Click "Continue" after setting the roles.

Once the service account is created, click on it to view its details.

Go to the "Keys" tab, click "Add Key", and choose "Create new key".

Choose "JSON" as the key type and click "Create". The JSON key file will be downloaded automatically. This file contains your API credentials.

Secure Your Credentials JSON File

Store the JSON File Securely: This file is the key to your Google Cloud resources, so keep it secure. Never commit this file to source control or expose it publicly.

Reference in Your Application: In your server script, reference this file in the code where you authenticate with Google APIs, typically using the keyFile property when initializing your Google API client.

Set Permissions and Folder ID

Go to Google Drive and find the folder you wish to use for uploads.

Right-click the folder and select "Share".

Enter the service account's email address and grant "Editor" permissions, then click "Send".

Replace 'your-drive-folder-id' in the 'server.js' file with the actual ID of the folder you want to use. The folder ID can be found in the folder's URL when viewing it in Google Drive.

#3

Running Your Application

****Start the Server**:** Run 'node server.js' to start your Express server.

****Open the Front-end**:** Open 'index.html' in a web browser to access your video capture application.

NETFLIX Update

Add-ons